

SEC101: SCIENTIFIC WRITING AND COMPUTATIONAL ANALYSIS TOOLS

M.Sc. Computer Science

Submitted By:

Harshit Kumar More

Roll No: 16

Department of Computer Science

Submitted To:

Bhomik Kaushik

November 22, 2025

Problem Statement: 01

Problem: You are given the matrix and vector. Perform following Operations:

- 1) Display the matrix A and vector b using MATLAB
- 2) Compute the determinant of Matrix A
- 3) Find the Inverse of Matrix A
- 4) Compute the Eigen values of Matrix A
- 5) Create new Matrix M=reshape(1:12,3,4) and perform the transpose and Horizontally concatenate M with a column of ones to form C=[M , 1] and display the result

Below is the MATLAB script:

```
1  % matrix_array_ops.m
2  % Matrix creation, indexing, solving linear systems, eigenvalues.
3
4  clc; clear; close all;
5  A = [4 1 2; 0 3 -1; 1 0 2];
6  b = [7; 4; 3];
7
8  % Display
9  disp('Matrix A:'); disp(A);
10 disp('Vector b:'); disp(b);
11
12 % Matrix operations
13 detA = det(A);
14 invA = inv(A);           % small matrix ok
15 x = A\b;                 % solve Ax = b (preferred)
16 eigVals = eig(A);
17
18 fprintf('det(A)=%g\n', detA);
19 disp('Inverse of A:'); disp(invA);
20 disp('Solution x = A\b:'); disp(x);
21 disp('Eigenvalues:'); disp(eigVals);
22
23 % Reshape and concatenate
24 M = reshape(1:12, 3, 4); % 3x4 matrix
```

```

25 M_t = M';           % transpose
26 C = [M, ones(3,1)]; % horizontal concat
27
28 disp('Example reshape M:'); disp(M);
29 disp("Transpose:"); disp(M_t)
30 disp("Horizontal Concat:"); disp(C)

```

Output:

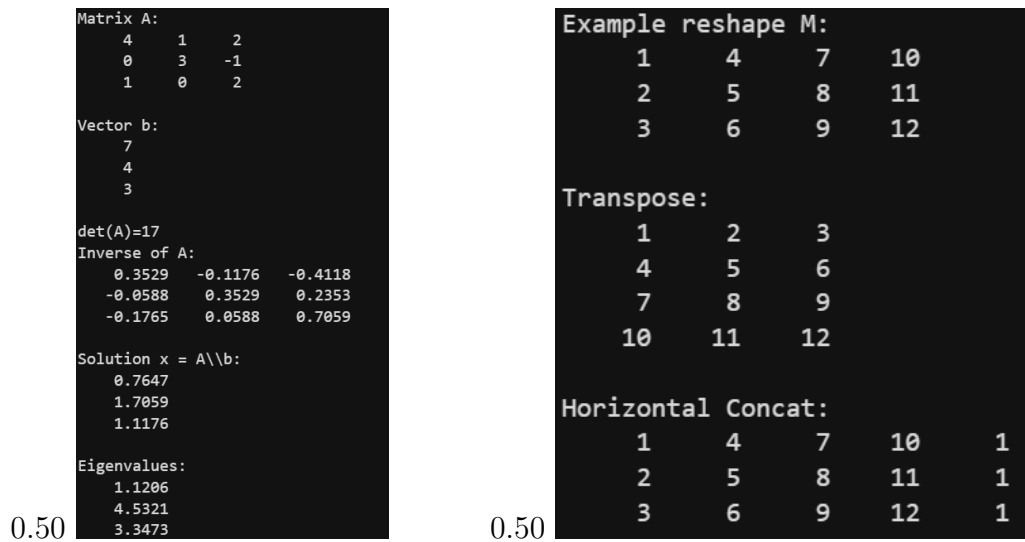


Figure 1: Output of Problem 1

Problem Statement: 02

Problem: Write a MATLAB script that demonstrates the basic concepts of scientific data visualization. Your task is to:

1. Generate a vector x over the interval $[0, 2\pi]$ and compute the functions:

$$y_1 = \sin(x), \quad y_2 = \cos(2x)e^{-0.3x}.$$

2. Create a figure window containing two subplots:

- **Subplot 1:** Plot $y_1 = \sin(x)$ using a line plot. Include a suitable title, axis labels, and enable the grid.
- **Subplot 2:** Create a scatter plot of y_2 , where the color of each point represents its value. Add title, labels, grid lines, and a colorbar.

3. Save the final plotted figure as a PNG file named `plot_examples.png`.

Below is the MATLAB script:

```
1 % plotting_visualization.m
2 % Demonstrates line plot, scatter, subplot, labels and saving figure.
3
4 clc; clear; close all;
5 x = linspace(0,2*pi,200);
6 y1 = sin(x);
7 y2 = cos(2*x).*exp(-0.3*x);
8
9 figure('Name','Plotting Examples','NumberTitle','off');
10
11 subplot(2,1,1)
12 plot(x,y1,'LineWidth',1.5)
13 title('y = sin(x)')
14 xlabel('x'), ylabel('sin(x)')
15 grid on
16
17 subplot(2,1,2)
18 scatter(x, y2, 8, y2) % color by value
19 title('y = cos(2x)*e^{-0.3x}')
20 xlabel('x'), ylabel('y2')
21 colorbar
```

```
22 grid on
23
24 % Save as PNG
25 saveas(gcf, 'plot_examples.png');
26 fprintf('Saved figure to plot_examples.png\n');
27
```

Output:

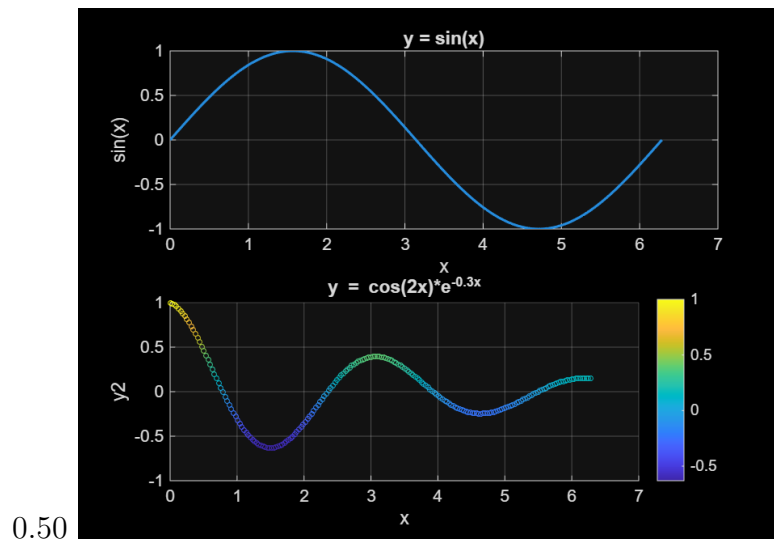


Figure 2: Output of Problem 2

Problem Statement: 03

Problem: Write a MATLAB script that demonstrates fundamental image processing operations using an input image of your choice. Your task is to:

1. Load an input image and display it as the **original image**.
2. Convert the image to **grayscale** only if it contains three color channels (i.e., RGB format).
3. Resize the grayscale image to **60% of the original size** for demonstration purposes.
4. Perform **contrast enhancement** using an appropriate intensity adjustment method.
5. Apply **Canny edge detection** to extract major edges from the enhanced image.
6. Use morphological operations:
 - **Dilation** to strengthen and connect edges,
 - **Erosion** to thin the edges back to a regular thickness.
7. Display the following results in a 2×2 subplot layout:
 - Grayscale (resized) image,
 - Contrast-adjusted image,
 - Canny edge map,
 - Morphologically cleaned edge image.

Below is the MATLAB script:

```
1 % image_processing_demo.m
2 % Use the uploaded screenshot, convert to grayscale, enhance and detect edges.
3
4 clc; clear; close all;
5 %clc → clears the Command Window
6 %clear → removes variables from workspace
7 %close all → closes all open figure windows
8
9
```

```
10 I = imread("doremon.png"); %Stores it as matrix I
11
12 figure('Name','Original Image'), imshow(I), title('Original');
13
14 % Convert to grayscale if needed
15 if size(I,3) == 3 % If the image has 3 color channels → it is RGB → convert to grayscale
16     Ig = rgb2gray(I);
17 else
18     Ig = I;
19 end
20
21 % Resize for demonstration
22 Ig_small = imresize(Ig, 0.6); %Shrinks image to 60% of original size
23
24 % Contrast enhancement
25 Ig_adj = imadjust(Ig_small); % imadjust stretches pixel intensity
26
27 % Edge detection
28 edges = edge(Ig_adj, 'Canny'); %Finds sharp intensity changes
29
30 % Morphological cleanup %Edges might be thin, broken, or noisy.
31 edges_clean = imdilate(edges, strel('disk',1)); %Dilation: thickens edges & connects b
32 edges_clean = imerode(edges_clean, strel('disk',1)); %Erosion: thins edges back to nor
33
34 % Display
35 figure('Name','Image Processing Results');
36 subplot(2,2,1), imshow(Ig_small), title('Grayscale (resized)');
37 subplot(2,2,2), imshow(Ig_adj), title('Contrast adjusted');
38 subplot(2,2,3), imshow(edges), title('Canny edges');
39 subplot(2,2,4), imshow(edges_clean), title('Cleaned edges');
40
41 % Save results
42 imwrite(Ig_adj, 'image_gray_adj.png');
43 imwrite(edges_clean, 'image_edges.png');
44 fprintf('Saved image_gray_adj.png and image_edges.png\n');
45
46
```

Output:

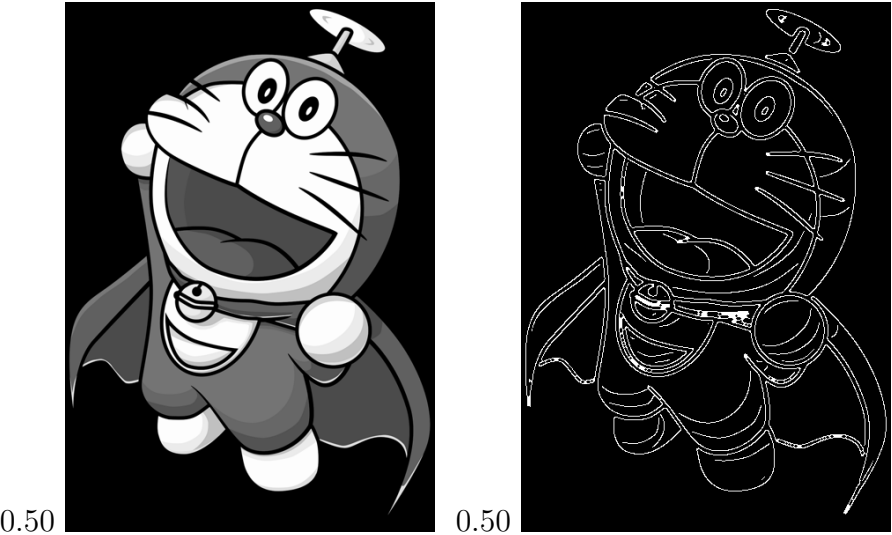
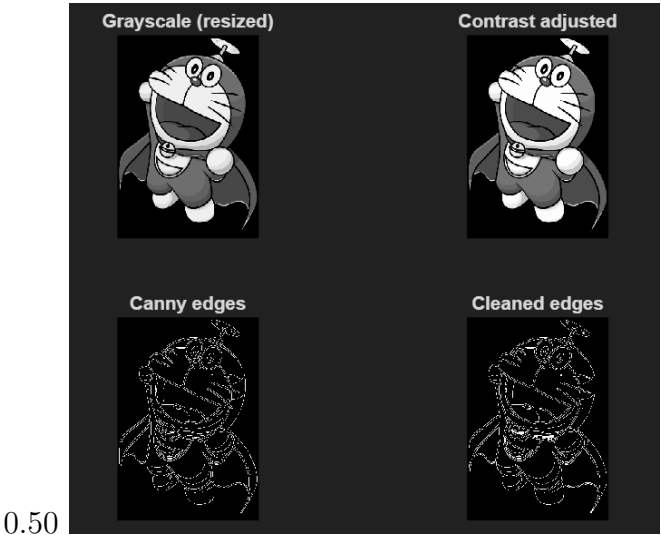


Figure 3: Output of Problem 3

Problem Statement: 04

Problem: Write a MATLAB script that converts a given input image into an emoji-based mosaic representation. Your task is to:

1. Read an input image and convert it to **grayscale**. Resize the image to a fixed size (e.g., 300×300) to standardize the processing.
2. Divide the grayscale image into non-overlapping blocks of equal size (e.g., 20×20 pixels).
3. For each block:
 - Compute the **average brightness** value (0–255),
 - Map this brightness level to a corresponding **emoji** selected from a predefined list of colored emojis, where darker emojis represent lower brightness and lighter emojis represent higher brightness.
4. Construct an **emoji-art matrix** in which each block of the grayscale image is replaced by its mapped emoji.
5. Convert the emoji matrix into a properly formatted **emoji text string** suitable for display.
6. Display the resulting emoji-based image representation inside a MATLAB **UI figure** using a **uitextarea** with an emoji-supported font.

Below is the MATLAB script:

```
1  clc; clear; close all;
2
3  img = imread("doremon.png");      % change file if needed
4  img_gray = rgb2gray(img);
5
6  img_gray = imresize(img_gray, [300 300]);
7
8
9  % Emoji mapping based on brightness
10 emoji_list = ["", "", "", "", "", "", "", ""];
11 num_emojis = length(emoji_list);
12
```

```
13 % Each emoji covers equal brightness range
14 range_step = 256 / num_emojis;
15
16
17 % Define block size
18 block_size = 20; % each emoji = block average
19 rows = size(img_gray, 1);
20 cols = size(img_gray, 2);
21
22 rows_b = rows / block_size;
23 cols_b = cols / block_size;
24
25 emoji_art = strings(rows_b, cols_b);
26
27
28 % Build emoji matrix
29 for r = 1:rows_b
30     for c = 1:cols_b
31
32         % extract block
33         block = img_gray((r-1)*block_size+1 : r*block_size, ...
34                         (c-1)*block_size+1 : c*block_size);
35
36         % compute average brightness 0{255
37         avg_brightness = mean(block(:));
38
39         % convert brightness to emoji index
40         idx = ceil(avg_brightness / range_step);
41         idx = max(1, min(num_emojis, idx));
42
43         emoji_art(r, c) = emoji_list(idx);
44     end
45 end
46
47
48 emoji_str = strjoin(join(emoji_art, ""), newline);
49
50
51 fig = uifigure('Name','Emoji Image Compressor', 'Position',[100 100 800 600]);
52 txt = uitextarea(fig, ...
53     'Value', emoji_str, ...
54     'FontName','Segoe UI Emoji', ...
55     'FontSize', 14, ...
```

```
56     'Position',[20 20 760 560]);  
57  
58 % Done!  
59 disp("Emoji compression completed!");  
60  
61  
62
```

Output:

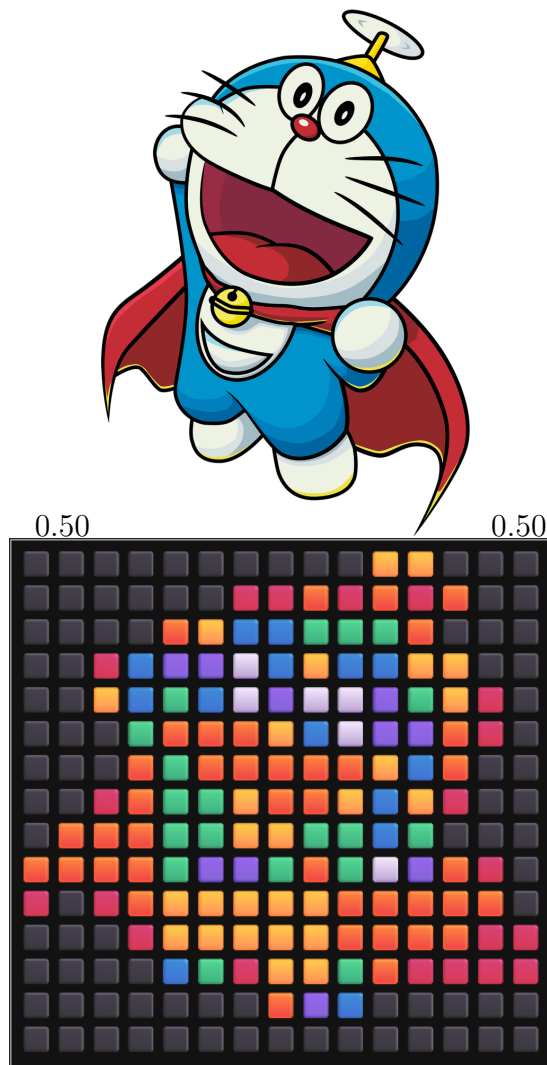


Figure 4: Output of Problem 4